

Tecnológico Nacional de México
Instituto Tecnológico de Mazatlán
Ingeniería en Sistemas Computacionales

Reporte de práctica

Comparación de delay() y millis() con Arduino

Arduino Uno R4 WiFi · 3 LEDs (verde, amarillo, rojo)

Dato	Detalle
Materia	Sistemas Programables
Práctica	Comparación de delay() y millis() con Arduino
Número de control	231000103
Placa	Arduino Uno R4 WiFi
Herramientas	Arduino IDE 2, Tinkercad Circuits, monitor serie
Fecha de pruebas	15 de septiembre de 2026
Fecha de entrega	16 de septiembre de 2026

Resumen

Se compararon dos formas de controlar el mismo circuito de tres LEDs (verde, amarillo y rojo) con un Arduino Uno R4 WiFi: una usando la función bloqueante `delay()` y otra usando la función no bloqueante `millis()`. Ambas versiones manejan los mismos intervalos por LED (500, 1000 y 1500 ms), pero solo la versión con `millis()` permite que los tres LEDs cambien de estado de forma independiente y que el programa realice otras tareas, como enviar un mensaje por el monitor serie, sin detenerse. El análisis del código y las pruebas en el armado físico confirmaron que, con `delay()`, los LEDs se encienden uno a la vez sin superposición, mientras que con `millis()` pueden coincidir encendidos varios LEDs al mismo tiempo.

I. Introducción

En un microcontrolador como Arduino, casi todos los programas necesitan controlar eventos que ocurren en momentos distintos: encender un LED cierto tiempo, leer un sensor cada intervalo, o revisar si llegó un dato. La forma más simple de manejar esos tiempos es la función `delay()`, que detiene la ejecución del programa durante los milisegundos indicados [1]. Sin embargo, esa misma simplicidad es su limitación: mientras el programa espera, no puede hacer nada más, ni siquiera revisar otro evento con un tiempo distinto. La función `millis()`, en cambio, devuelve el tiempo transcurrido desde que se encendió la placa sin detener el programa [2], lo que permite construir temporizadores independientes para cada evento. Esta práctica compara ambos enfoques usando el mismo circuito de tres LEDs con tres intervalos distintos, para mostrar de forma concreta la diferencia entre un programa bloqueante y uno no bloqueante.

II. Objetivos

Objetivo general. Comparar el comportamiento de un programa bloqueante (con `delay()`) y uno no bloqueante (con `millis()`) controlando el mismo circuito de tres LEDs con distintos intervalos de tiempo.

- Controlar tres salidas digitales con `delay()`, estableciendo un tiempo de encendido distinto para cada LED.
- Controlar las mismas tres salidas con `millis()`, usando una variable de “tiempo anterior” y el operador de resta para detectar cuándo cambiar cada LED sin bloquear el programa.
- Enviar un mensaje por el monitor serie desde dentro de la versión no bloqueante, sin depender de `delay()`.
- Documentar y comparar el comportamiento de ambas versiones mediante un diagrama de tiempo derivado del código.

III. Marco teórico

A. Temporización bloqueante con `delay()`

`delay(ms)` pausa la ejecución del programa durante el número de milisegundos indicado [1]. Internamente, el microcontrolador simplemente espera sin ejecutar ninguna otra instrucción del programa del usuario, por lo que ninguna otra parte del código —ni siquiera una lectura de botón o un mensaje por el monitor serie— puede ocurrir mientras dura la espera. Cuando un programa necesita manejar varios eventos con distintos tiempos, como los tres LEDs de esta práctica, `delay()` obliga a atenderlos uno tras otro, sumando sus tiempos de espera.

B. Temporización no bloqueante con `millis()`

La función `millis()` devuelve, en un entero sin signo, la cantidad de milisegundos transcurridos desde que el programa empezó a ejecutarse, y no detiene el flujo del programa [2]. La técnica estándar para usarla —empleada también en el ejemplo oficial `Blink Without Delay` [3]— consiste en guardar en una variable el valor de `millis()` en el momento del último cambio y, en cada vuelta del `loop()`, restar ese valor al tiempo actual: si la diferencia es mayor o igual al intervalo deseado, se actualiza el evento y se guarda el nuevo tiempo. Repitiendo esta comparación para cada LED con su propia variable, es posible manejar varios intervalos de forma independiente dentro del mismo `loop()`, sin que ninguno bloquee a los demás.

IV. Materiales y herramientas

Tabla I. Materiales y herramientas.

Elemento	Uso en la práctica
Arduino Uno R4 WiFi	Ejecuta ambos programas y genera las señales digitales hacia los LEDs
3 LEDs (verde, amarillo, rojo)	Indicadores visuales, cada uno con su propio intervalo de tiempo
3 resistencias de 330 Ω	Limitan la corriente que pasa por cada LED
Protoboard	Permite armar el circuito sin soldar los componentes
Cables Dupont	Conexión entre el Arduino y la protoboard
Arduino IDE 2	Programación de la placa y lectura del monitor serie
Tinkercad Circuits	Elaboración del diagrama de conexiones

V. Desarrollo

A. Circuito

Los tres LEDs se conectaron en la protoboard, cada uno en serie con una resistencia de 330 Ω para limitar la corriente, con el cátodo hacia la tierra común y el ánodo hacia el pin digital correspondiente del Arduino [4], [5]. El mismo circuito, sin ninguna modificación, se usó tanto para la versión con `delay()` como para la versión con `millis()`; solo cambia el programa cargado en la placa. La Figura 1 muestra el diagrama elaborado en Tinkercad Circuits, y la Tabla II detalla las conexiones.

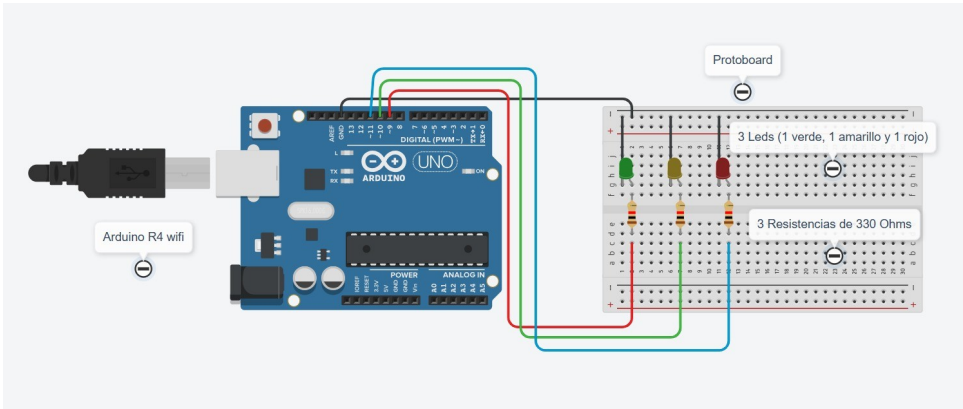


Figura 1. Diagrama de conexiones (Tinkercad Circuits).

Tabla II. Conexiones del circuito.

LED	Conexión	Función
Verde	Pin 9 del Arduino	Parpadea cada 500 ms
Amarillo	Pin 10 del Arduino	Parpadea cada 1000 ms
Rojo	Pin 11 del Arduino	Parpadea cada 1500 ms; envía un mensaje por el monitor serie al encender (solo en la versión con millis)

B. Programa

Versión con delay(). El programa enciende el LED verde durante 500 ms, lo apaga, enciende el amarillo durante 1000 ms, lo apaga, enciende el rojo durante 1500 ms, lo apaga, y vuelve a repetir la secuencia. Como cada delay() bloquea el programa, los tres LEDs jamás están encendidos al mismo tiempo: cada uno debe esperar a que termine el anterior.

Versión con millis(). El programa guarda en tres variables (tiempoVerde, tiempoAmarillo, tiempoRojo) el momento del último cambio de cada LED, y en cada vuelta del loop() compara millis() contra cada una. Cuando la diferencia alcanza el intervalo correspondiente (500, 1000 o 1500 ms), el LED cambia de estado y se actualiza su variable de tiempo. Como los tres intervalos se revisan en la misma vuelta del ciclo sin ningún delay(), los LEDs cambian de estado de forma independiente y pueden coincidir encendidos. Además, cada vez que el LED rojo se enciende, el programa envía por el monitor serie (9600 baudios) el mensaje «Ximena la mas chambeadora», algo que la versión con delay() no puede hacer sin interrumpir su secuencia.

VI. Resultados

El comportamiento de ambas versiones se determinó a partir del código fuente y se verificó grabando el circuito en video: uno para la versión con delay() y otro para la versión con millis(), ambos disponibles en Video/Readme.txt del repositorio de la práctica. La Figura 2 muestra el armado físico usado en las dos pruebas.

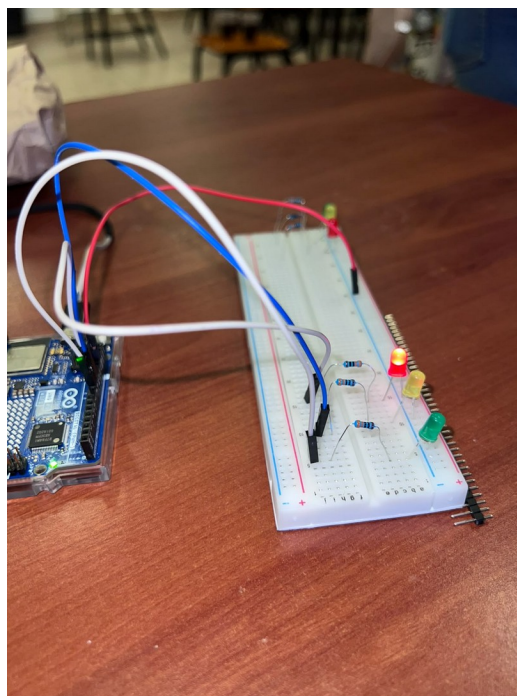


Figura 2. Armado físico con el Arduino Uno R4 WiFi y los tres LEDs.

Para comparar de forma precisa el comportamiento temporal de ambos programas, se calculó, a partir de la lógica de cada uno, el estado (encendido o apagado) de cada LED durante una ventana de 3000 ms. La Figura 3 corresponde a la versión con `delay()` y la Figura 4 a la versión con `millis()`; la Tabla III resume los intervalos de encendido de cada LED en ambas versiones.

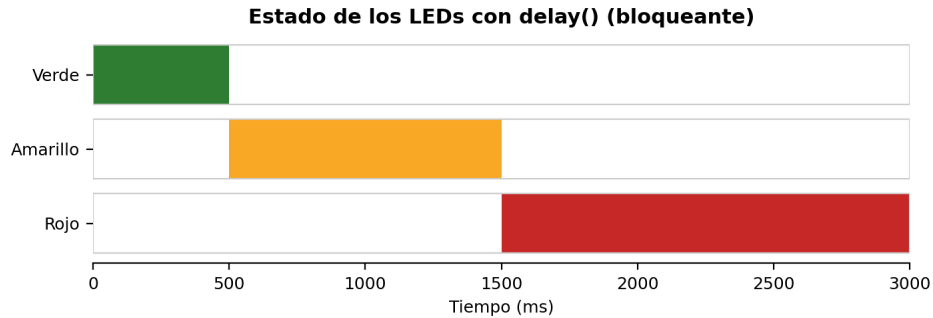


Figura 3. Estado de los LEDs con `delay()`, calculado a partir del código (ventana de 3000 ms).

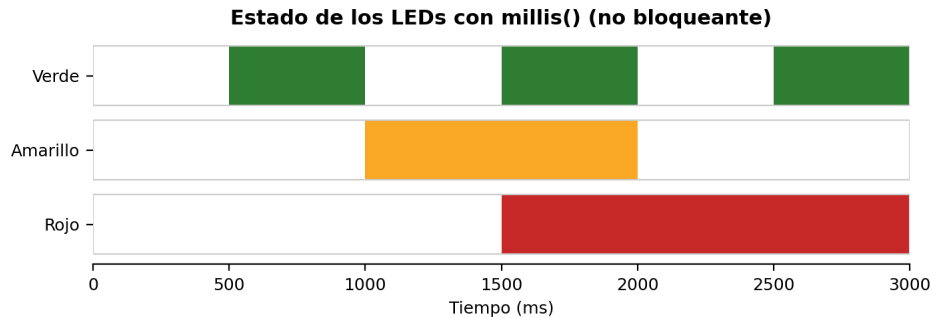


Figura 4. Estado de los LEDs con `millis()`, calculado a partir del código (ventana de 3000 ms).

Tabla III. Intervalos de encendido de cada LED, por versión del programa.

Versión	LED	Encendido dentro de 0–3000 ms
delay()	Verde	0–500 ms
delay()	Amarillo	500–1500 ms
delay()	Rojo	1500–3000 ms
millis()	Verde	500–1000, 1500–2000, 2500–3000 ms
millis()	Amarillo	1000–2000 ms
millis()	Rojo	1500–3000 ms

Las gráficas completas y el detalle de esta comparación se encuentran en Resultados/Resultados_practica3.pdf.

VII. Análisis de resultados

Comportamiento bloqueante. Como muestra la Figura 3, con `delay()` nunca hay más de un LED encendido al mismo tiempo: el ciclo completo dura exactamente 3000 ms —la suma de los tres `delay()`— y se repite

siempre en el mismo orden (verde, amarillo, rojo). Esto confirma, de forma directa en el circuito, que `delay()` detiene por completo el programa mientras espera: no hay manera de que otro LED cambie de estado durante esa espera porque el microcontrolador no ejecuta ninguna otra instrucción.

Comportamiento no bloqueante. La Figura 4 muestra que, con `millis()`, los tres LEDs cambian de estado en su propio intervalo sin esperarse entre sí. En la ventana analizada, el LED verde y el LED rojo coinciden encendidos entre 2500 y 3000 ms —algo que la versión con `delay()` nunca permite—, lo que demuestra que el programa revisa y actualiza los tres intervalos dentro de la misma vuelta del `loop()` sin bloquear ninguno de ellos.

Tarea adicional durante la espera. Solo la versión con `millis()` pudo enviar el mensaje «Ximena la mas chambeadora» por el monitor serie exactamente cuando el LED rojo se enciende, sin alterar el resto de la secuencia. Reproducir ese mismo comportamiento con `delay()` habría requerido colocar el `Serial.println()` en un punto fijo del código, sin poder garantizar que ocurriera justo en el instante de encendido de un LED mientras los otros también cambian de estado.

Costo de cada enfoque. La comparación también deja ver el costo de cada técnica: la versión con `delay()` es más corta y fácil de leer, pero solo sirve cuando los eventos pueden atenderse estrictamente uno después de otro. La versión con `millis()` requiere una variable de tiempo y una comparación por cada evento independiente, lo que hace el código más largo, pero es la única de las dos que escala a más eventos simultáneos sin perder capacidad de respuesta.

VIII. Conclusiones

Conclusión general. Se cumplió el objetivo de comparar un programa bloqueante y uno no bloqueante controlando el mismo circuito. El análisis del código y el diagrama de tiempo construido a partir de él mostraron con claridad que `delay()` obliga a atender los eventos en secuencia, mientras que `millis()` permite manejar varios intervalos de forma independiente dentro del mismo ciclo del programa. La prueba adicional de enviar un mensaje por el monitor serie solo en la versión con `millis()` confirmó que la técnica no bloqueante deja al programa libre para hacer otras tareas mientras cuenta el tiempo.

Conclusión individual (L231000103). En esta práctica entendí la diferencia entre `delay()` y `millis()` no solo en teoría sino viéndola en el mismo circuito: con el mismo cableado, cambiar únicamente el programa cambió por completo el comportamiento de los LEDs. Lo que más me quedó claro fue por qué `millis()` necesita una variable de tiempo por cada evento —sin ella no hay forma de saber cuánto ha pasado desde el último cambio sin detener el programa—. También noté que agregar el mensaje del monitor serie fue sencillo en la versión con `millis()` y hubiera sido más difícil de ubicar correctamente en la versión con `delay()`. Como mejora, agregaría un cuarto LED con su propio intervalo para comprobar que la técnica de `millis()` sigue funcionando sin modificar la estructura del programa, algo que con `delay()` habría alargado aún más la secuencia.

Referencias

- | | | | | | | |
|---|---|----------------------------|----------|------------|------------|-----|
| [1] | Arduino, | « <code>delay()</code> », | Language | Reference. | Disponible | en: |
| https://www.arduino.cc/reference/en/language/functions/time/delay/ | | | | | | |
| [2] | Arduino, | « <code>millis()</code> », | Language | Reference. | Disponible | en: |
| https://www.arduino.cc/reference/en/language/functions/time/millis/ | | | | | | |
| [3] | Arduino, | «Blink Without Delay», | ejemplo | oficial. | Disponible | en: |
| https://docs.arduino.cc/built-in-examples/digital/BlinkWithoutDelay/ | | | | | | |
| [4] | Arduino, «UNO R4 WiFi», documentación oficial. Disponible en: https://docs.arduino.cc/hardware/uno-r4-wifi | | | | | |

[5] Arduino, «digitalWrite()», Language Reference. Disponible en:
<https://www.arduino.cc/reference/en/language/functions/digital-io/digitalwrite/>